

torch.nn.utils.rnn.pad_packed_sequence

https://pytorch.org/docs/stable/generated/torch.nn.utils.rnn.pad_packed_sequence.html

Description:

Pad a packed batch of variable length sequences. It is an inverse operation to `pack_padded_sequence()`. The returned Tensor's data will be of size $T \times B \times *$ (if `batch_first` is `False`) or $B \times T \times *$ (if `batch_first` is `True`), where T is the length of the longest sequence and B is the batch size.

Function Signature

```
torch.nn.utils.rnn.pad_packed_sequence(sequence,  
batch_first=False, padding_value=0.0, total_length=None)  
[source]#
```

Parameters

Returns

Return type

Returns:

tuple[torch.Tensor, torch.Tensor]

Code Examples

```
>>> from torch.nn.utils.rnn import pack_padded_sequence,
pad_packed_sequence
>>> seq = torch.tensor([[1, 2, 0], [3, 0, 0], [4, 5, 6]])
>>> lens = [2, 1, 3]
>>> packed = pack_padded_sequence(
...     seq, lens, batch_first=True, enforce_sorted=False
... )
>>> packed
PackedSequence(data=tensor([4, 1, 3, 5, 2, 6]),
batch_sizes=tensor([3, 2, 1]),
               sorted_indices=tensor([2, 0, 1]),
               unsorted_indices=tensor([1, 2, 0]))
>>> seq_unpacked, lens_unpacked = pad_packed_sequence(packed,
batch_first=True)
>>> seq_unpacked
tensor([[1, 2, 0],
        [3, 0, 0],
        [4, 5, 6]])
>>> lens_unpacked
tensor([2, 1, 3])
```

Notes & Warnings

NOTE

Note `total_length` is useful to implement the pack sequence -> recurrent network -> unpack sequence pattern in a Module wrapped in `DataParallel`. See this FAQ section for details.

Detailed Documentation

Documentation

Pad a packed batch of variable length sequences.

It is an inverse operation to `pack_padded_sequence()`.

The returned Tensor's data will be of size $T \times B \times *$ (if `batch_first` is `False`) or $B \times T \times *$ (if `batch_first` is `True`), where T is the length of the longest sequence and B is the batch size.

`total_length` is useful to implement the pack sequence -> recurrent network -> unpack sequence pattern in a Module wrapped in `DataParallel`. See this FAQ section for details.

`sequence` (`PackedSequence`) – batch to pad

`batch_first` (bool, optional) – if `True`, the output will be in $B \times T \times *$ format, $T \times B \times *$ otherwise.

`padding_value` (float, optional) – values for padded elements.

`total_length` (int, optional) – if not `None`, the output will be padded to have length `total_length`. This method will throw `ValueError` if `total_length` is less than the max sequence length in sequence.

Tuple of Tensor containing the padded sequence, and a Tensor containing the list of lengths of each sequence in the batch. Batch elements will be re-ordered as they were ordered originally when the batch was passed to `pack_padded_sequence` or `pack_sequence`.

`tuple[torch.Tensor, torch.Tensor]`